

Assignment 2: Build, Trigger, Observe, and Explain Blocking & Signals

Linux Systems and Performance - Week 2

Signal-Aware Shell -> Interruptible Kernel Wait -> EVENT / SIGNAL / TIMEOUT

1. Assignment Philosophy

This assignment is not evaluated only by whether the code runs. You will build signal-aware and blocking mechanisms, intentionally trigger different wake-up conditions, observe the actual process/kernel behavior, and explain why each result occurred.

BUILD -> TRIGGER -> OBSERVE -> ANALYZE -> EXPLAIN

Important: AI coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

2. Challenge Levels

Level	Build	Trigger	Observe / Validate	Main Concept
Level 1	Signal-aware shell	background exit, Ctrl-C	strace + process state	SIGCHLD, SIGINT, waitpid(WNOHANG)
Level 2	event_wait / event_notify syscalls	EVENT or signal	blocked state + syscall return	wait queue, interruptible sleep, wake-up
Level 3	timeout + stored event_count	EVENT / SIGNAL / TIMEOUT / notify-before-wait	controlled experiment matrix	event state vs. wake-up notification

Levels are cumulative. The required experiments use controlled scenarios. General multicore synchronization and a production-quality event subsystem are outside the scope of this assignment.

3. Common Interface and Rules

- Extend the Week 1 myshell implementation. The existing cd, pwd, exit, external-command, and pipeline behavior must continue to work.
- Use C for the required implementation unless otherwise approved.
- Level 2 and Level 3 use custom Linux system calls plus small external user-space commands. Do not hide the required behavior inside the shell.
- Busy waiting is prohibited. A waiting task must actually sleep in the kernel.
- Record the exact commands used for every required experiment. Screenshots alone are not sufficient; include short interpretations.
- Document kernel-version-dependent details when they affect the implementation or observed behavior.
- If AI tools are used, include a short AI-use statement and identify at least one suggestion/code issue that you verified, changed, or rejected.

4. Level 1 - Signal-Aware Shell

4.1 Required Implementation

- Support background execution using '&'. The parent shell must return immediately to the prompt.
- Reap terminated background children using SIGCHLD and waitpid(..., WNOHANG), or an equivalent correct design.
- Handle Ctrl-C / SIGINT so that an interactive foreground command can be interrupted without unintentionally terminating myshell.
- Full bash-style job control (fg/bg, terminal process groups, etc.) is not required.

4.2 Required Functional Test

```
myshell> sleep 5 &
[1] 4215
myshell> echo responsive
responsive
...
[1] Done  sleep 5
myshell> sleep 30
^C
myshell> echo shell-survived
shell-survived
```

4.3 Required Observation

Run myshell under strace and preserve child-process behavior:

```
strace -f -o signal.trace ./myshell
```

From signal.trace and your process observations, identify evidence for:

Evidence	What to Find	What It Demonstrates
Signal setup	rt_sigaction(...) / related signal setup	myshell installs signal behavior
Background child exit	SIGCHLD delivery	child termination is asynchronous
Child reaping	wait4/wait-family call with WNOHANG	shell reaps without blocking the prompt
Foreground Ctrl-C	SIGINT delivery and foreground child termination/interruption	shell survives while foreground work is interrupted

4.4 Required Explanation

1. Why can myshell display another prompt immediately after starting a background process?
2. Why is SIGCHLD asynchronous with respect to the shell's normal command-processing loop?
3. Why is waitpid(..., WNOHANG) useful in SIGCHLD-based child reaping?
4. What would happen if terminated background children were never reaped?
5. Why must Ctrl-C interrupt the foreground command without terminating myshell itself?

5. Level 2 - Blocking Event with Custom System Calls

5.1 Required Programs and Kernel Interface

Implement two external commands that are launched through the normal myshell fork/exec path:

```
event_wait.c    -> ./event_wait
event_notify.c  -> ./event_notify
```

```
long event_wait(void);
long event_notify(void);
```

The external commands call the custom system calls. This keeps the execution path visible: myshell -> fork -> exec -> syscall -> kernel wait/wake-up -> syscall return.

5.2 Required Kernel Behavior

- ./event_wait calls event_wait(). If no event is available, the task must sleep in the kernel; busy waiting is prohibited.
- Use an interruptible waiting mechanism such as wait_event_interruptible().
- ./event_notify calls event_notify(), makes the event condition true, and wakes the waiting task using wake_up_interruptible().
- Run ./event_wait as a background command so myshell remains interactive.
- ./event_wait must install a SIGINT handler (without SA_RESTART) or use an instructor-approved equivalent so an interruptible wait can be observed returning because of a Unix signal.

5.3 Required Experiment A - EVENT Wake-Up

```
myshell> ./event_wait &
[2] 5301
Waiting for event...
myshell> ./event_notify
Event notified.
Event received.
[2] Done
```

While ./event_wait is blocked, record its process state using a system observation command such as:

```
ps -o pid,ppid,state,wchan:24,comm -p <PID>
```

Expected observation: the task is normally in an interruptible sleeping state while waiting and consumes little CPU. The exact WCHAN text is kernel/configuration dependent and may be unavailable on some systems.

5.4 Required Experiment B - SIGNAL Interruption

```
myshell> ./event_wait &
[3] 5350
Waiting for event...
myshell> kill -INT 5350
Wait interrupted by signal.
```

Record how the syscall returns at user level (for example, an EINTR-style result) and compare it with the normal EVENT path. Kernel-internal restart codes or exact strace text may depend on the signal-handler configuration and kernel version.

5.5 Required Analysis

6. Why does ./event_wait consume little CPU while it is blocked?
7. What is the difference between making an event condition true and merely waking a sleeping task?
8. How is event_notify() different from sending a Unix signal?
9. Why can an interruptible wait return because of a signal even when no event was generated?
10. How is this mechanism similar to a blocking device-driver read path?

6. Level 3 - EVENT, SIGNAL, TIMEOUT, and Stored Events

6.1 Extended Interface

```
long event_wait(int timeout_ms);
long event_notify(int count);
```

```
./event_wait <timeout_ms>
./event_notify <count>
```

- Use an interruptible timeout-based wait mechanism such as `wait_event_interruptible_timeout()`.
- `event_wait(timeout_ms)` must distinguish EVENT, SIGNAL/INTERRUPTED, and TIMEOUT.
- Replace a simple boolean flag with `event_count`. Each successful wait consumes one stored event.
- `event_notify(count)` adds `count` events to `event_count` and wakes waiting task(s) as appropriate.
- The required grading experiments update `event_count` in controlled scenarios. General concurrent notifier synchronization is out of scope.

6.2 Required Stored-Event Semantics

```
event_notify(2)      -> event_count = 2
event_wait(5000)     -> EVENT immediately; event_count = 1
event_wait(5000)     -> EVENT immediately; event_count = 0
event_wait(5000)     -> no stored event; task sleeps
```

Key concept: `wake_up_interruptible()` does not store an event. The separately maintained `event_count` preserves notifications that occur before a waiter begins sleeping.

6.3 Required Experiment Matrix

Case	Trigger	Required Observation	Explanation Focus
A. EVENT	wait first, then notify 1	waiter returns EVENT	condition became true + wake-up
B. SIGNAL	wait first, then SIGINT	waiter returns interrupted result	interruptible sleep + signal
C. TIMEOUT	wait; do not notify	waiter returns TIMEOUT	timeout expiration
D. NOTIFY BEFORE WAIT	notify 2 before any wait	first 2 waits return immediately; third blocks/times out	stored event_count

6.4 Optional Advanced Experiment - Multiple Waiters

Optional: place multiple `event_wait` processes on the same wait queue and observe how notifications are consumed. Exact FIFO wake ordering is not required. If you attempt this extension, document the synchronization assumptions or primitives used to protect `event_count`.

7. Connection to Blocking Device I/O

This Assignment	Blocking Device-I/O Analogy
<code>./event_wait -> syscall</code>	<code>read(fd, ...)</code> enters the kernel
interruptible wait	no data available -> task sleeps on a wait queue
<code>./event_notify -> syscall</code>	device/event side makes data/event available
<code>wake_up_interruptible()</code>	driver/event path wakes sleeping task
EVENT / SIGNAL / TIMEOUT	blocking operation can finish for different reasons

8. Evidence Required

Level	Implementation Evidence	Runtime / Observation Evidence	Analysis Evidence
1	signal-aware myshell	SIGCHLD / WNOHANG + Ctrl-C trace	foreground/background + reaping
2	<code>event_wait</code> / <code>event_notify</code> syscalls	sleep state + EVENT vs SIGNAL return	wait queue + interruptible wake-up
3	timeout + <code>event_count</code>	4-case experiment matrix	wake-up vs stored event state

9. Submission Package

- Updated Week 1 myshell source code and Makefile.
- event_wait.c and event_notify.c user-space programs.
- For Level 2/3: kernel patch/diff or clearly identified list of modified kernel files.
- README with kernel version/architecture, build/boot/run instructions, event semantics, and highest completed level.
- Technical report (recommended 3-5 pages) organized by completed level.
- Selected strace/process-state evidence and the exact commands used. Do not submit pages of unannotated raw output.
- For Level 3: one table containing the EVENT, SIGNAL, TIMEOUT, and NOTIFY-BEFORE-WAIT experiments with observed results and explanations.
- A short AI-use statement if AI tools were used.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	35%	correct shell, custom syscall, blocking, timeout, and event semantics
Experiment & Observation	30%	controlled triggers, trace/state evidence, reproducibility
Technical Explanation	25%	correct reasoning about signals, sleep/wake-up, event state, and syscall return
Code Quality & Report	10%	error handling, clarity, kernel-version notes, concise evidence

A program that appears to work but is not experimentally validated or correctly explained cannot receive full credit. The quality of the controlled experiment and the explanation matters more than producing a particular screenshot.

11. Final Check-Off Questions

- Show a background child terminating and identify the SIGCHLD and nonblocking reap evidence.
- Predict what happens to myshell and the foreground hild before pressing Ctrl-C, then verify it.
- Show ./event_wait sleeping and explain why it is not busy waiting.
- Before running ./event_notify, predict exactly why the waiter should return.
- Interrupt a blocked waiter with SIGINT and explain why this is not the same mechanism as event_notify().
- Call ./event_notify 2 before any waiter exists, then explain why two later waits can return immediately.
- Explain why wake_up_interruptible() alone cannot implement notify-before-wait semantics.

Build it. Trigger it. Observe what actually happened. Explain why.